

CORE_STATE — Canonical ядро «Мозочок»

Статус: ЗАФІКСОВАНО (canonical checkpoint)

Призначення документа: Цей файл фіксує поточний стан ядра (decision + pattern memory) як **еталонний**. Усі подальші зміни дозволені **тільки поверх цього стану** або з можливістю повного відкату.

1. Загальна філософія

Мозочок — це **локальна, детермінована система прийняття торгових рішень**, яка: - не використовує онлайн-ШІ - не "вгадує", а **вирішує** - завжди може пояснити, **чому угода була дозволена або заблокована**

Ядро розділене на логічні шари:

1. **Decision Engine** — прийняття рішення (ГОТОВО)
 2. **Pattern Memory** — накопичення досвіду (СТАБІЛЬНО)
 3. **Input Layer** (PDF / image / chart) — НЕ входить у це ядро
 4. **Transport (Telegram / UI)** — НЕ входить у це ядро
-

2. Decision Engine — ФІНАЛЬНИЙ СТАН

2.1 Призначення

Decision Engine відповідає **лише** за одне:

Чи має право система виконати торгову дію в даний момент.

Він **не аналізує графік і не навчається сам**.

2.2 Вхідні дані

```
get_signal(  
    description: str,  
    base_signal: "UP" | "DOWN",  
    regime: str,  
    session: str  
)
```

- `description` — семантичний опис патерну
- `base_signal` — початковий намір (intent)
- `regime` — TREND / FLAT / etc

- `session` — LONDON / NY / ASIA
-

2.3 Вихід Engine

Engine завжди повертає **повну структуру стану**:

- `signal` — фінальний сигнал або NO_TRADE
 - `intent` — початковий намір (НЕ змінюється)
 - `decision` — ключове рішення ядра
 - `mode` — BOOTSTRAP | LIVE
 - `learning` — True / False
 - `confidence`
 - `authority`
 - `expectancy`
 - `decision_score`
 - `readiness`
 - `reason`
 - `notes[]`
-

2.4 Decision enum (зафіксовано)

Decision	Значення
<code>LEARN_BOOTSTRAP</code>	Навчання без торгів
<code>BLOCK_BOOTSTRAP</code>	Заборона через малу статистику
<code>BLOCK_AUTHORITY</code>	Недостатня довіра
<code>BLOCK_COOLDOWN</code>	Тимчасове блокування
<code>ALLOW_TRADE</code>	Дозвіл на угоду

2.5 Поведінка (важливо)

- BOOTSTRAP ≠ помилка
- NO_TRADE ≠ негативний сигнал
- BLOCK_ — це усвідомлене рішення*, а не відсутність логіки

Decision Engine **ніколи не форсує угоду**.

3. Pattern Memory — СТАБІЛЬНИЙ СТАН

3.1 Роль

Pattern Memory: - зберігає патерни - накопичує wins / losses - формує `authority` - може заморозувати (FROZEN)

Вона **не** приймає рішень.

3.2 Структура патерну

```
{
  "id": "uuid",
  "description": "...",
  "regime": "TREND",
  "wins": 0,
  "losses": 0,
  "avg_win": 1.0,
  "avg_loss": 1.0,
  "authority": 0.0,
  "state": "BOOTSTRAP",
  "freeze_until": 0
}
```

3.3 Поточні правила

- BOOTSTRAP → ACTIVE після ≥ 3 outcomes
- FROZEN при серії збитків + низькій authority
- Автоматичний вихід з freeze

⚠ Pattern Memory НЕ оптимізується без реальних outcomes

4. Поточний прогрес (реалістичний)

Компонент	Готовність
Decision Engine	100%
Pattern Memory (core)	85%
Learning loop (real data)	30%
Input analysis	0%
UI / Telegram	0%

Загальна готовність ядра: ~65%

Це стабільні 65%, без майбутніх "відкатів назад".

5. ЩО ЗАБОРОНЕНО РОБИТИ ЗАРАЗ

✗ Переписувати decision_engine ✗ Оптимізувати pattern_memory без статистики ✗ Додавати ML / нейронки ✗ Плутати input layer з decision

6. ДОЗВОЛЕНІ НАСТУПНІ КРОКИ

✓ Симуляція outcomes (WIN / LOSS) ✓ Batch-тести decision ✓ Логи та аналітика поведінки

Цей файл є точкою відкату. Будь-які зміни — тільки з новим CORE_STATE.md